

A Gazebo model of the Slider satellite platform

Frank Fourlas
Robotics & AI Group
Luleå University of Technology
Luleå, Sweden
fragkiskos.fourlas@ltu.se

Avijit Banerjee
Robotics & AI Group
Luleå University of Technology
Luleå, Sweden
jakub.haluska@ltu.se

Jakub Haluska
Robotics & AI Group
Luleå University of Technology
Luleå, Sweden
jakub.haluska@ltu.se

George Nikolakopoulos
Robotics & AI Group
Luleå University of Technology
Luleå, Sweden
george.nikolakopoulos@ltu.se

Abstract—In this technical report, a complete simulation model of a 2D satellite platform is presented. The model is made to simulate the open-source Slider Platform that was designed by the Robotics and Artificial Intelligence group of Luleå University of Technology. It is fully compatible with the dimensions, inertial characteristics and control methods of the original Slider. In addition, a plugin has been developed that can be loaded on the model and allow the user to realistically control its virtual thrusters. A detailed analysis of the software developed for this project as well as instructions and example of use are included in this report.

Index Terms—simulation, satellite, open-source, model, plugin, gazebo, ROS

I. INTRODUCTION

Simulation is an integral part of any robotics project. It allows the engineer to test his ideas and software in a risk-free environment before investing into fragile and expensive equipment or risking their damage. It also allows for fast changes of the environment and experimental parameters, a key feature for early-stage experimentation.

Many companies and organizations understand this need and provide the public with simulation models for their robotic systems. Notable examples include the TurtleBot 3 and the Pioneer 3-AT [1], [2]. Other platforms, such as Boston Dynamic's Spot, keep their project proprietary and researchers rely on third-party models, adaptations and equivalents for their simulations. *SpotMicro*, the *robotics_spot* controller by the SoftServe robotics group and *champ* are some of the open-source projects dedicated on simulating Spot [3]–[5].

This project aims to offer the community an open-source solution for simulating 2D, floating platforms used in laboratory environments to experiment with the dynamics of satellites in space. It builds on top of the Slider platform, a robotic system designed by B. Avijit, J. Haluska, S.G. Satpute, et al. for the Robotics and Artificial Intelligence group of Luleå University of Technology [6].

Researchers around the world use modern, state-of-the-art and industry standard frameworks for simulation and communication. For this reason, Gazebo [7], the simulation environment by Open Robotics, was chosen for this project. The controller plugin is written using ROS2 [8] libraries and the *gazebo_ros* interface for message passing. This enables most researchers to use the model without changing their setup and ensures long term compatibility with the latest versions of ROS2. The model introduced in this report is available with an open-source licence on [GitHub](#).

This technical report is split in four parts. section II will summarize the technical characteristics of the slider platform and the development of the gazebo robot model. Sections III and IV explain the setup and operation of the model with ROS2 and the thruster selection logic respectively. Finally, section V will present a proof of correct operation of the model.

II. THE SLIDER PLATFORM MODEL

A. Dimensions and Inertial Characteristics

As previously mentioned, this model is based on the open-source design "Slider". The dimensions that are important for this project are described in the blueprints of fig. 1. Regarding the collision geometry of the model explained in section II-C, the bounding cylinder of Slider has a radius $R = 195.8\text{mm}$ and a length $L = 200.3\text{mm}$.

Regarding the inertial characteristics of the platform, it has a mass $m = 4.436\text{kg}$ and its moment of inertia around the Z-axis is $I_{ZZ} = 1.092\text{kg m}^2$. The full matrix of moments of inertia for 3D motion are filled in the model using the results of CAD simulations of Slider provided by the original paper's authors. Only I_{ZZ} is important though, since the model is only capable of 2D motion, and the rest are used only for the sake of completion.

B. Thruster Poses and Thrust

There is a total of eight thrusters on the platform. These are ON/OFF thrusters, meaning they cannot produce

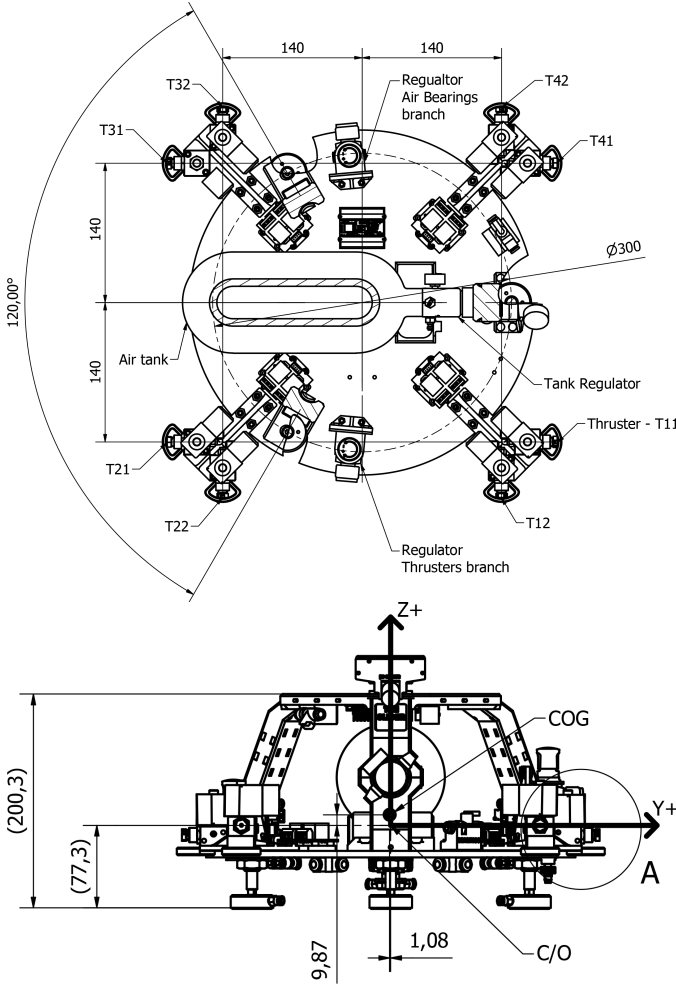


Fig. 1. Parts of the Slider blueprint containing important dimensions and thruster names

TABLE I
THRUSTER POSITION & ORIENTATION

Thruster	Position (mm)	Orientation (deg)
T ₁₁	(195, -140)	0
T ₁₂	(140, -195)	270
T ₂₁	(-195, -140)	180
T ₂₂	(-140, -195)	270
T ₃₁	(-195, 140)	180
T ₃₂	(-140, 195)	90
T ₄₁	(195, 140)	0
T ₄₂	(195, 195)	90

variable thrust. At any given time they can be either outputting their maximum thrust or be completely turned off. Table I describes the positions and orientations of each thruster on the slider body as seen in figs. 1 and 2.

Each thruster is capable of producing a maximum of $F_{max} = 0.7\text{N}$. Due to their position, each thruster also produces $\tau_{max} = 0.7 \cdot 140 \times 10^{-3} = 98 \times 10^{-3}\text{N m}$ of torque on either direction depending on their orientation. Table II explains which thrusters need to be powered to move the platform in standard directions. From this table,

it is apparent that the total force that can be applied in each direction is $F_{total} = 2 \cdot 0.7 = 1.4\text{N}$ and the total torque is $\tau_{total} = 4 \cdot 98 \times 10^{-3} = 392 \times 10^{-3}\text{N m}$.

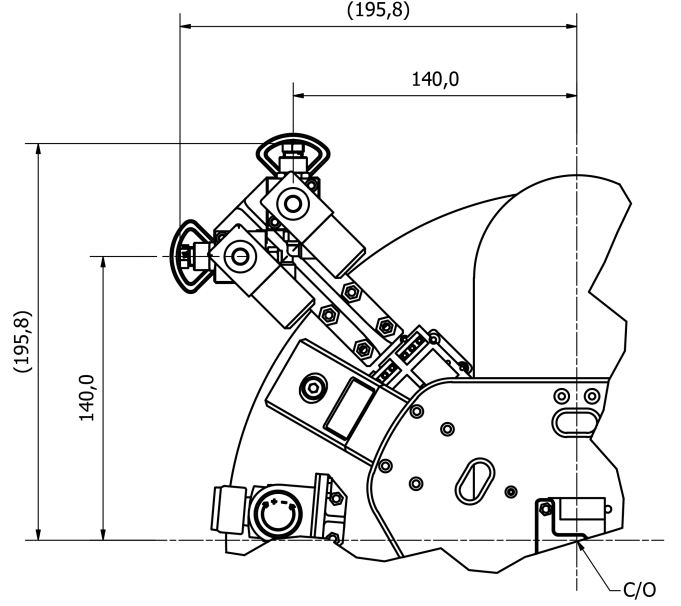


Fig. 2. Part of the Slider blueprint containing distance of thrusters from COG

C. Robot Description

ROS uses an XML specification to describe robots called the Unified Robotics Description Format (URDF). URDF is a structural language that describes robotic links and joints, their properties and relations. Xacro (XML Macros) is a macro language used in combination with URDF in order to create shorter and more readable XML files using variables and macros. In contrast to ROS, Gazebo, uses neither of these standard languages. It uses the Simulation Description Format (SDF), which, in addition to robotic links and joints, can describe environments, physics, visualization and control. A Python library is provided as a ROS2 package that can translate URDF/xacro code, infused with special <gazebo> tags, to SDF.

In this project, the robot description exists in the \model directory. The main model file, \model\slider.urdf.xacro defines the slider's body and thruster links. For the sake of simplicity and simulation speed, the collision element of the model is defined as the bounding cylinder of the model, as described in section II-A. The model's visual is a low-polygon mesh render of the Slider provided as a Collada file in this directory. The \model\dimensions.xacro and \model\gazebo.xacro files, contain some model dimensions and simulation parameters and plugins respectively.

III. SENSORS AND CONTROL

Gazebo allows the designer to attach plugins to SDF objects to expand on the simulation capabilities. The types of plugins that can be used on URDF robots are model, sensor and visual plugins [9]. Model plugins affect the links and joints of a robot, while visual plugins affect the model's image. Finally, sensor plugins attach to `sensor` tags and allow the model to communicate how it perceives its surroundings. The reader is referred to the project's [GitHub repository](#) for detailed instructions on compiling and installing the plugin described in section III-B as well as a full example of the model's operation.

A. Sensors

In an experimental setup, precise knowledge of the robots pose, velocity and acceleration is essential. Even in real-world applications, robots can estimate part of their odometry using sensors such as IMU, GNSS and cameras using visual odometry algorithms [10]. To simulate this, the P3D model plugin of the `gazebo_ros_plugins` package is used. This plugin can be configured in the file `\model\gazebo.xacro` to match the engineers needs (e.g. adding simulated noise).

The model is also configurable to use a camera. The lines of code that create a camera frame, sensor and appropriate sensor plugin in the URDF files are commented out. You can use the instructions included on the `README` file to enable it in your project.

B. Controller Plugin

The objective of any computer model is the accurate simulation of the exact operation of the real robot. For this reason, each virtual thruster of the model needs to be individually controllable and only have binary states (ON/OFF). A custom model plugin is written in C++ to enable this kind of particular control of the model.

The plugin uses the `gazebo_ros` library to open a ROS node that subscribes to the `\slider\slider_controller` topic. This topic expects an 8-value boolean array in the form of a `UInt8MultiArray` message. Each element of the array represents an ON or OFF flag for each thruster from T_{11} to T_{42} (explained in fig. 1). For example, the user can run the following command to turn ON thrusters T_{12} and T_{31} and turn OFF the rest:

```
ros2 topic pub --once \slider\slider_controller
std_msgs\msg\UInt8MultiArray "{data:[0,1,0,0,0,0,0,0]}"
```

Note that if a thruster is commanded ON, it will stay ON until a relative OFF flag is received.

On every simulation cycle, for every thruster that currently has an ON flag, a force and torque is applied to the model relative to its own coordinate frame. To calculate the forces and torques that need to be applied, each thruster is assigned a force direction vector $\vec{F}_{XY}$ and torque direction vector $\vec{\tau}_{XY}$ depending on its pose on the

platform. For example, T_{32} , applies a force on the $+Y$ axis and a *CCW* torque so its thrust direction vectors are $\vec{F}_{32} = [0, 1, 0]$ and $\vec{\tau}_{32} = [0, 0, 1]$ (see table I). Finally, the force and torque applied to the model for each thruster are $F_i = F_{max} * \vec{F}_{XY}$ and $\tau_i = \tau_{max} * \vec{\tau}_{XY}$.

IV. THRUSTER SELECTION LOGIC

A. TSL

Slider is an over-actuated system, since it can perform six distinct maneuvers (forward, backward, left, right, clockwise and counter-clockwise) but is equipped with eight thrusters. This means that there are more than one ways to perform a specific maneuver. For example applying -50% τ_{total} (CW) can be done by either applying 50% power to actuators T_{11} , T_{22} , T_{31} and T_{42} or 100% power only to T_{11} and T_{31} . To tackle this issue, a slightly complex thruster selection logic (TSL) is used.

First of all, eq. (1) explains the relation between F_x , F_y and τ and the force applied by each thruster.

$$A\mathbf{x} = b \quad (1)$$

Where $b = [F_x, F_y, \tau]^T$

$$A = [A_1 \dots A_8]$$

$$\text{With } A_i = \begin{bmatrix} \cos(\beta_{AB}) \\ \sin(\beta_{AB}) \\ (r_{TAB}^y \cos(\beta_{AB}) - r_{TAB}^x \sin(\beta_{AB})) \end{bmatrix}$$

With r_{TAB}^x , r_{TAB}^y and β_{AB} being the position and orientation of the thrusters (found in table I) and $\mathbf{x}$ is a column vector of the forces applied by each thruster.

The problem of determining the thrust that should be applied by each thruster to achieve a given force-torque command in body coordinates is reduced to an optimization problem as follows:

$$\begin{aligned} \min_x \quad & J = x^T \cdot x \\ \text{s.t.} \quad & Ax = b \\ & 0 \leq x \leq x_{max} \end{aligned} \quad (2)$$

The solution to this optimization problem is the optimal thruster output that provides the target force and torque while minimizing the actuation effort and respecting the hardware limitations.

TABLE II
THRUSTER ACTIVATION LOGIC

Direction	Thrusters							
	T_{11}	T_{12}	T_{21}	T_{22}	T_{31}	T_{32}	T_{41}	T_{42}
Forward			✓		✓			
Backward	✓						✓	
Left		✓		✓				
Right						✓		✓
Clockwise	✓			✓	✓			✓
C-Clockwise		✓	✓			✓	✓	

A solution of this optimization problem is included in the project's repository using the Optimization Engine

(OpEn) library [11]. OpEn is a framework for generating optimization solvers for non-linear problems in Rust using the PANOC optimizer [12]. The author of OpEn also published interfaces to access the generated optimizers from Python and MATLAB. The user is referred to the project's **README** file for instructions on preparing the optimizer for use.

B. PWM

The output of the prementioned TSL is a continuous, decimal value. This of course is incompatible with Slider's ON/OFF thrusters. To simulate a continuous value thrust, a Pulse-Width Modulation (PWM) signal is used to drive the thrusters. PWM signals are square waves with a fixed frequency and variable duty cycle, meaning that per cycle, the percentage of the period the signal is high changes. High frequency PWM signals can be used to control the energy delivery to a system capable of only binary states.

A ROS2 node that implements a PWM generator for each thruster, with configurable frequency and resolution, is provided in this project's repository as part of the examples. The node subscribes to the `\2befilled\topic` and accepts 8-value, percentage arrays as input, in the form of `Float32MultiArray` messages. Each element of the array represents the duty cycle of the PWM signal for each thruster from T_{11} to T_{42} . In turn, the node sends commands to the controller plugin, turning each thruster ON and OFF, simulating a PWM signal with the configured frequency, resolution and duty cycle for each thruster. For example, the user can use the following command to turn thrusters T_{11} and T_{42} on at 30% when the PWM node is running.

```
ros2 topic pub --once \2befilled\topic std_msgs\
msg\Float32MultiArray "{data:[0.3,0.0,0.0,
0.0,0.0,0.0,0.0,0.0,0.3]}"
```

V. PROOF OF NORMAL OPERATION

Tests were made to confirm the normal operation of the model. For this purpose, all the combinations for standard movements of the platform described in table II were tested and provided the expected results. Results can be found in graphs in fig. 3. These data can be found in the form of rosbag recordings in the GitHub repository of the project.

CITING THIS WORK

This open source model is published under the MIT License. However, if you use the model, we would appreciate it that you cited it accordingly. For academic publications, you can cite as follows:

REFERENCES

- [1] Turtlebot 3. [Online]. Available: <https://www.turtlebot.com/turtlebot3/>
- [2] Pioneer 3-at - ros robots. [Online]. Available: <https://robots.ros.org/pioneer-3-at>
- [3] Spotmicro - github. [Online]. Available: <https://github.com/mike4192/spotMicro>

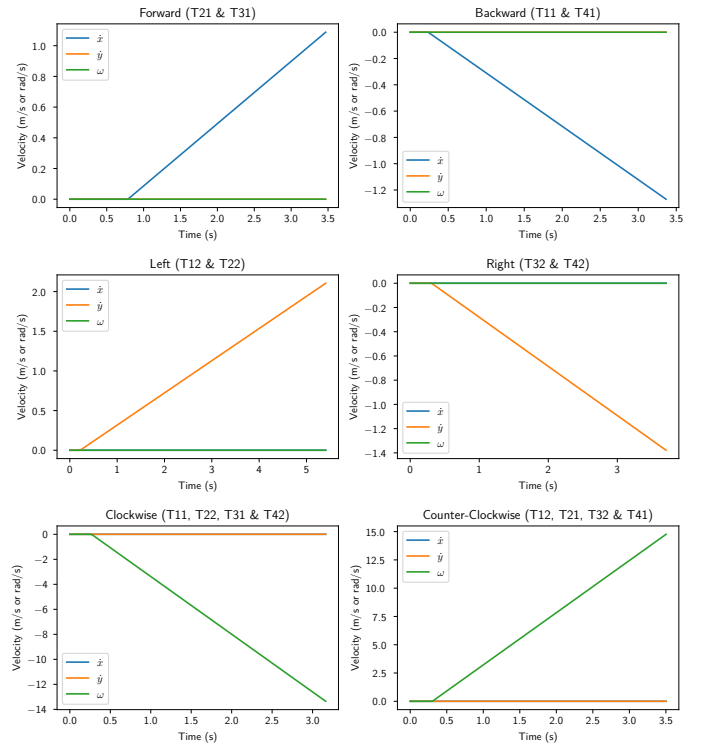


Fig. 3. Model acceleration under specific thruster configurations.

- [4] robotics_spot - github. [Online]. Available: https://github.com/SoftServeSAG/robotics_spot
- [5] champ - github. [Online]. Available: <https://github.com/chvmp/champ>
- [6] A. Banerjee, J. Haluska, S. G. Satpute, D. Kominiak, and G. Nikolakopoulos, "Slider: On the design and modeling of a 2d floating satellite platform," *CoRR*, vol. abs/2101.06335, 2021. [Online]. Available: <https://arxiv.org/abs/2101.06335>
- [7] Gazebo. [Online]. Available: <http://gazebo.org/>
- [8] Robot Operating System (ROS). [Online]. Available: <http://www.ros.org/>
- [9] Gazebo Plugins in ROS. [Online]. Available: https://classic.gazebo.org/tutorials?tut=ros_gzplugins
- [10] R. Mur-Artal, J. M. M. Montiel, and J. D. Tardós, "ORB-SLAM: a versatile and accurate monocular SLAM system," *IEEE Transactions on Robotics*, vol. 31, no. 5, pp. 1147–1163, 2015.
- [11] P. Sopasakis, E. Fresk, and P. Patrinos, "OpEn: Code generation for embedded nonconvex optimization," in *IFAC World Congress*, Berlin, Germany, 2020.
- [12] L. Stella, A. Themelis, P. Sopasakis, and P. Patrinos, "A simple and efficient algorithm for nonlinear model predictive control," in *IEEE Conference on Decision and Control (CDC)*, Dec 2017, pp. 1939–1944.